ECS (ELASTIC CONTAINER SERVICE)

ECS (Elastic Container Service)

A fully managed container orchestration service that makes it easy to deploy, manage and scale docker containers.

It helps you:

- Deploy Docker containers
- Manage containers
- Scale applications automatically
- Run containers on EC2 or AWS Fargate

Key Features

- Fully managed by AWS
- Supports Docker containers
- Highly available
- Secure
- Integrates with many AWS services

ECS COMPONENTS

1. Cluster
   It's a group of resources and tasks where containers are run

2. Service
   keeps desired number of tasks are running and ensures high availability

3. Tasks definition
   The blueprint of your application where u define containers, cpu, memory, port, environment variables etc.

4. Task
   A running instance of container based on task definition.

BENEFITS OF ECS

1. Fully managed service
2. Easy deployment
3. High availability

4. Auto Scaling
5. Secure
6. Pay only for resources used
7. Deep AWS integration
8. Supports microservices
9. Supports CI/CD pipelines

USE CASES

1. Microservices
2. Web Applications
3. REST APIs
4. Backend Applications
5. Batch Processing
6. CI/CD Deployments
7. Machine Learning Applications

ECS WORKFLOW

Developer

Build Docker Image

Push Image to Amazon ECR

Create Task Definition

Create ECS Cluster

Create ECS Service

Run Tasks

Application Load Balancer

▼

Users Access Application

|

▼

CloudWatch Monitoring

PRACTICAL ON ECS (ELASTIC CONTAINER SERVICE)

1.CREATE A ECR (elastic container registry)

2.CREATE A IMAGE / TAG & PUSH TO ECR

1. vim index.html
   <h1> These is my version-1 </h1>
2. docker build -t testing .
3. docker tag testing:latest (ur ECR tags)
4. docker push

1.CREATE A CLUSTER

1. cluster name ------> MyCluster
2. namespace --------> MyCluster
3. compute ----------> Fargate only
4. create a cluster

2.CREATE A TASK DEFINATION (BLUEPRINT)

1. task name ----------> MyTask
2. launch type ----------> AWS Fargate
3. container name -----> my-container
4. image URI ----------> (ECR repository URL)
5. container port -------> 80
6. log collection --------> use log collection enabled
7. Health Check --------> CMD-SHELL, curl -f http://localhost/ || exit 1
8. create task definition

3.CREATE A SERVICE IN CLUSTER

1. click on cluster (MyCluster)
2. create a service
3. select Task definition --------> MyTask
4. Task definition revision -----> 1
5. service name --------> my-svc
6. compute option ----> Capacity provider strategy
7. Capacity provider --> Fargate
8. desired tasks --------> 2
9. networking
   1. vpc ----------> default vpc
   2. subnets ----> default subnets

3. sg ----------> my-sg

10. public-ip -------> turned on
11. load balancing
    1. vpc ------> default vpc
    2. load balancing type ----> Application Load Balancer
    3. load balancer name ----> my-lb
    4. change the listener and target group according to application
12. Auto scaling
    1. Minimum number of tasks --------> 2
    2. Maximum number of tasks -------> 10
    3. Scaling policy type --------> target tracking
    4. policy name ----------> increase policy
    5. ECS service metric ---> ECSServiceAverageCPUUtilization
    6. Target Value ----------> 70
13. create

4.CLICK ON SERVICE

1. Check all the status of the containers/tasks

5.CLICK ON LOAD BALANCER

1. go to ec2
2. click on the load balancer
3. copy & paste the DNS name on browser

6.APPLYING ROLLING OUT STRATEGY

1. changes in the index.html file
   <h1> These is my version-2 </h1>
2. docker build -t testing .
3. docker tag testing: latest (ur ECS tags)
4. docker push

5.TO UPDATE THE SERVICE

1. click on the service
2. click on force new deployment
3. update

6.CHECK THE SERVICE

1. u get the new version of the deployment
